

COMPARING SHORTEST PATH ALGORITHMS ON GRID MAPS

Jazib Waqas | Hunain Abbas | Salman Adnan

INTRODUCTION

Objective: To analyze how effecient Jump Point Search (JPS4) actually is and to determine the ideal environments where it outperforms standard algorithms.

○ DJIKSTRA

Explores every reachable cell in order of distance from start. Always correct, but has no sense of where the goal is, so it essentially maps the entire reachable area first.

○ A*

Adds Manhattan distance to the goal as a lower-bound estimate. Picks the cell with the lowest total estimated cost at each step, so the search moves toward the goal instead of spreading evenly

○ JPS4

Skips over long open corridors entirely. Stops only at jump points, places where an obstacle forces a real routing choice. Only those cells enter the open list, so the list stays small.

All three guarantee the same shortest path. The difference is how many cells each one has to look at.

METHODOLOGY



IMPLEMENTATION & APPROACH

- Live Game: Built a custom Snake game to watch the algorithms work in real-time.
- Visual Footprint: Added a tracker so we can actually see every single tile the algorithm investigates.

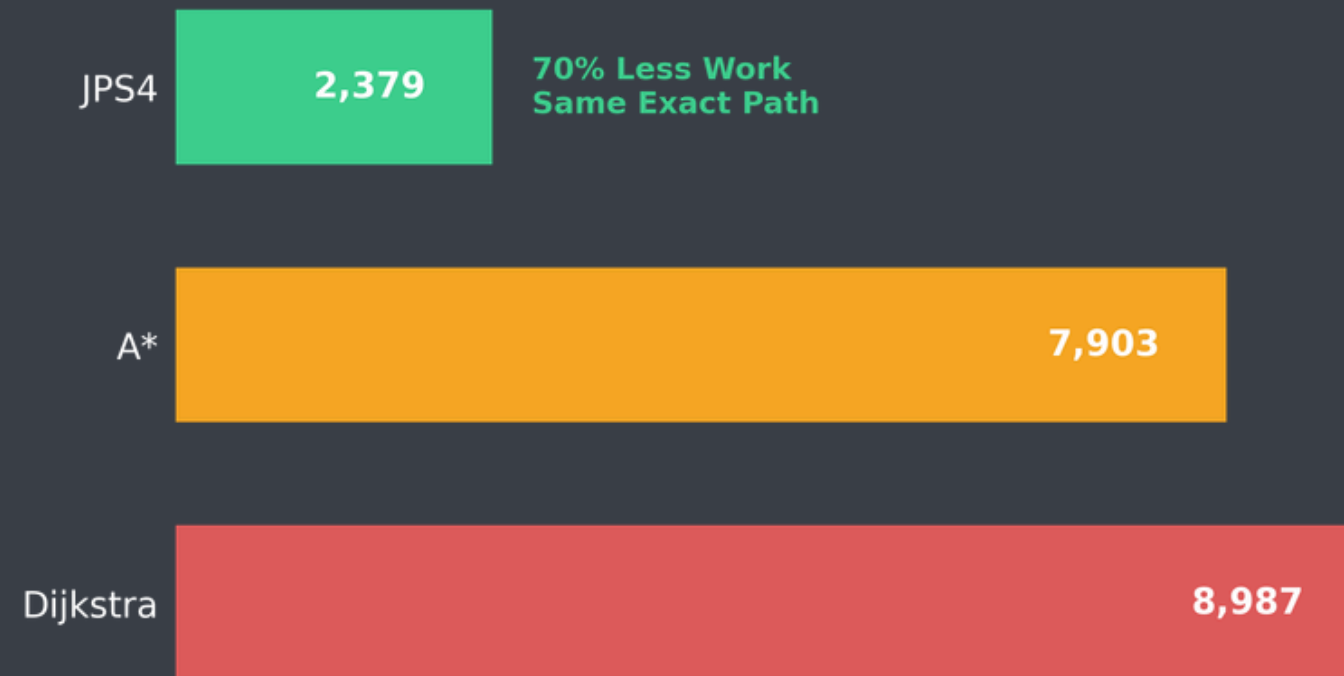


EVALUATION SETUP

- Grid Sizes: Tested on medium (50x50) and large (100x100) maps.
- Obstacles: Tested 7 different clutter levels, from mostly empty (10%) to very crowded mazes (40%).
- What We Measured: We measured "Tiles Checked" (Expansions) instead of just a timer. It is the fairest way to measure the true workload.

RESULTS

Tiles Checked to Find Optimal Path
(100x100 Grid, Sparse)



Benchmark Results (420 Automated Trials):

- **In Open Maps (10% Obstacles):** JPS4 absolutely dominates. It only has to check 30% of the tiles that A* does. The fewer obstacles there are, the more it can "jump."
- **In Crowded Mazes (40% Obstacles):** The gap shrinks because the jumps are blocked by walls. However, JPS4 still wins, doing about 40% less work than A*.
- **100% Accuracy Guaranteed:** Across all 420 trials on both grid sizes, JPS4 never missed the optimal route. It skips thousands of tiles, but the final path is exactly identical to A*.

**THANK
YOU**